

Van der pol oscillator

Amara Pranav

*Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aid23003@bl.students.amrita.edu*

V. Sidharth

*Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aid23054@bl.students.amrita.edu*

B G Rajath Siddarth

*Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aid23006@bl.students.amrita.edu*

Paruchuri Sai

*Department of Artificial Intelligence
Amrita School of Artificial Intelligence, Bengaluru
Amrita Vishwa Vidyapeetham, India
bl.en.u4aid23063@bl.students.amrita.edu*

Abstract—The Van der Pol oscillator is a nonlinear dynamical system widely used for studying self-sustained oscillations, nonlinear damping, and limit cycle behavior. In this work, the Van der Pol oscillator was modeled and simulated in MATLAB Simulink using an analog computing inspired setup to analyze its dynamic behavior across multiple nonlinear regimes. A comprehensive dataset generation framework was developed by varying the nonlinear parameter μ across harmonic, weak nonlinear, strong nonlinear, and relaxation oscillation regions. Oscillator displacement, velocity, and acceleration signals were extracted from Simulink simulations and processed through validation, transient removal, sliding-window segmentation, normalization, and statistical feature extraction. Features such as RMS, entropy, crest factor, zero-crossing rate, kurtosis, and signal energy were computed to characterize oscillatory dynamics. The final dataset consisted of 16,514 samples generated from multiple nonlinear regimes. Multiple machine learning models including Ridge Classifier, Logistic Regression, Linear Discriminant Analysis (LDA), Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Random Forest, Extra Trees, Gradient Boosting, Gaussian Naive Bayes, and Multi-Layer Perceptron (MLP) architectures were evaluated for oscillation regime classification. Hyperparameter optimization was performed using the Grid Search optimization algorithm. Among all evaluated models, the Ridge Classifier achieved the best performance with an accuracy of 99.70%, demonstrating that the extracted statistical and nonlinear features effectively capture the underlying dynamics of the Van der Pol oscillator. The proposed work demonstrates the integration of nonlinear dynamical systems, analog computation, signal processing, and machine learning for intelligent oscillation regime analysis.

Index Terms—Van der Pol Oscillator, Nonlinear Dynamics, Limit Cycle, Analog Computing, Simulink, Feature Extraction, Machine Learning, Oscillation Classification, Nonlinear Oscillations

I. INTRODUCTION

Nonlinear dynamical systems are widely used to model complex phenomena in electrical engineering, physics, biology, and communication systems. Unlike linear systems, nonlinear systems can exhibit behaviors such as self-sustained oscillations, limit cycles, bifurcations, and relaxation oscillations.

These properties make nonlinear oscillators important for understanding complex system dynamics and stability.

One of the most widely studied nonlinear oscillators is the Van der Pol oscillator, originally introduced by Balthasar Van der Pol while analyzing oscillations in vacuum tube circuits. The oscillator was developed to explain nonlinear electrical behavior observed in circuits containing active components. Over time, the Van der Pol equation became a classical mathematical model for studying nonlinear damping and periodic oscillatory behavior.

The Van der Pol oscillator is derived from a modified RLC circuit where the passive resistor is replaced by a nonlinear active element. In such systems, energy is injected into the oscillator at low amplitudes and dissipated at higher amplitudes, resulting in stable oscillations with finite amplitude. This nonlinear energy exchange mechanism distinguishes the Van der Pol oscillator from conventional damped harmonic oscillators.

The governing equation of the Van der Pol oscillator is given by:

$$y'' + \mu(y^2 - 1)y' + y = 0 \quad (1)$$

where μ is the nonlinear damping parameter that controls the behavior of the system. For smaller values of μ , the oscillator exhibits nearly sinusoidal oscillations similar to linear harmonic motion. As the value of μ increases, nonlinear effects become dominant, leading to distorted waveforms and relaxation oscillations characterized by slow and rapid transitions.

Due to its rich nonlinear behavior, the Van der Pol oscillator has applications in analog circuits, biological rhythm modeling, synchronization systems, signal processing, and nonlinear control systems. The oscillator is also frequently used as a benchmark system for studying phase-plane analysis, limit cycles, and nonlinear time-domain responses.

In recent years, machine learning techniques have increasingly been used for analyzing nonlinear dynamical systems

and identifying oscillatory patterns automatically from signal data. Feature extraction methods help convert raw oscillatory signals into meaningful numerical representations that capture waveform characteristics, energy variations, and nonlinear behavior. Such approaches enable efficient classification and analysis of different oscillatory regimes without relying solely on manual observation.

In this work, the Van der Pol oscillator was modeled and simulated using MATLAB and Simulink, and its behavior was analyzed for different values of the nonlinear parameter μ . A data-driven framework was further explored for oscillation regime analysis using signal-processing features and machine learning techniques. The major contributions of this work are summarized as follows:

- Modeling and simulation of the Van der Pol oscillator using MATLAB/Simulink.
- Analysis of oscillator behavior across multiple nonlinear regimes by varying the parameter μ .
- Generation of a labeled oscillatory dataset from simulated system responses.
- Extraction of statistical and nonlinear signal features for oscillation characterization.
- Comparative evaluation of multiple machine learning models for oscillation regime classification.

II. PROPOSED METHODOLOGY

The proposed methodology consists of six major stages: analog modeling of the Van der Pol oscillator, dataset generation, feature extraction, exploratory data analysis, machine learning preprocessing, and classification model training. The overall workflow combines nonlinear dynamical system simulation with data-driven machine learning analysis.

A. Analog Setup of Van der Pol Oscillator

1) *Initial Analog Setup Without Scaling*: The Van der Pol oscillator originates from a modified nonlinear RLC circuit in which the conventional passive resistor is replaced by a nonlinear active element. The original RLC circuit consists of an inductor (L), capacitor (C), resistor (R), and an external voltage source. Applying Kirchhoff's Voltage Law (KVL) to the circuit gives:

$$LI' + RI + \frac{Q}{C} = E \quad (2)$$

where:

- L = inductance,
- R = resistance,
- C = capacitance,
- Q = charge stored in the capacitor,
- $I = \frac{dQ}{dt}$ = current flowing through the circuit,
- E = external voltage source.

Differentiating the equation with respect to time yields:

$$LI'' + RI' + \frac{I}{C} = 0 \quad (3)$$

This equation resembles the classical damped harmonic oscillator equation:

$$my'' + dy' + ky = 0 \quad (4)$$

where the resistor continuously dissipates energy from the system, causing the oscillations to gradually decay over time.

To generate self-sustained oscillations, Van der Pol replaced the passive resistor with a nonlinear active element whose voltage-current relationship depends on the current magnitude. The nonlinear damping term is modeled as:

$$R(I) = I^2 - \alpha \quad (5)$$

where α represents the threshold current level. For small current values, the nonlinear element injects energy into the system, whereas for larger current values, it dissipates energy. This balance between energy injection and damping produces stable periodic oscillations.

Substituting the nonlinear damping term into the circuit equation gives:

$$LI'' + (I^2 - \alpha)I' + \frac{I}{C} = 0 \quad (6)$$

Rearranging and normalizing the equation leads to the standard Van der Pol differential equation:

$$y'' + \mu(y^2 - 1)y' + y = 0 \quad (7)$$

where μ is the nonlinear damping parameter controlling the strength of nonlinearity in the oscillator.

The equation contains three important components:

- y'' : acceleration term,
- $\mu(y^2 - 1)y'$: nonlinear damping term,
- y : restoring force term.

The nonlinear damping term governs the oscillatory behavior of the system. When $|y| < 1$, the quantity $(y^2 - 1)$ becomes negative, causing energy to be added into the system and increasing oscillation amplitude. When $|y| > 1$, the damping becomes positive, removing energy from the system and stabilizing the oscillations. As a result, the system naturally converges to a stable limit cycle independent of the initial conditions.

The initial analog computer setup, as shown in Fig. 1 directly implements this nonlinear differential equation using interconnected integrators, summation blocks, gain blocks, and nonlinear multipliers.

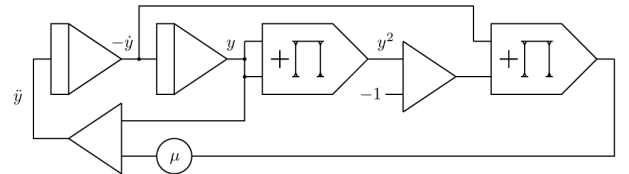


Fig. 1. Van der pol oscillator

The signal $\ddot{y}$ is first integrated to obtain $-\dot{y}$, and a second integration produces the displacement signal y . The feedback structure then reconstructs the nonlinear damping term of the Van der Pol equation.

The multiplier block generates the nonlinear term y^2 from the displacement signal. A summer subtracts the constant value 1 to produce the term $(y^2 - 1)$. This output is multiplied with the velocity term $\dot{y}$, thereby realizing the nonlinear damping expression $(y^2 - 1)\dot{y}$.

The parameter block μ controls the strength of nonlinear damping and scales the feedback contribution. The resulting signal is then combined with the restoring force term y and fed back into the integrator chain to generate the acceleration signal $\ddot{y}$.

The feedback loop continuously regulates the oscillation amplitude. For small amplitudes, negative damping injects energy into the system, while for large amplitudes, positive damping dissipates energy. This dynamic balance causes the oscillator to converge toward a stable limit cycle.

Although the unscaled setup accurately represents the Van der Pol dynamics, direct implementation can lead to excessively large signal amplitudes and instability in numerical simulation. Therefore, scaling techniques are introduced before practical Simulink implementation.

2) *Scaling of the Van der Pol Equation:* Direct implementation of the Van der Pol oscillator in an analog computer or Simulink environment can lead to excessively large signal amplitudes, improper gain distribution, and numerical instability. Therefore, scaling techniques are required before practical implementation. In this work, two types of scaling were applied:

- Amplitude scaling
- Time scaling

The parameter μ controls the overall nonlinear behavior of the oscillator and is assumed to lie within the interval $[0, 20]$. Scaling the Van der Pol program is nontrivial because the nonlinear parameter μ significantly affects the signal amplitudes and feedback dynamics.

a) *Amplitude Scaling:* Through experimentation and observation of the oscillator response, it was found that the displacement signal approximately satisfies:

$$-2 \leq y \leq 2 \quad (8)$$

To constrain the signal within the normalized interval $[-1, 1]$, the displacement variable was scaled as:

$$\hat{y} = \frac{y}{2} \quad (9)$$

Hence,

$$y = 2\hat{y} \quad (10)$$

The first derivative becomes:

$$\frac{dy}{dt} = 2 \frac{d\hat{y}}{dt} \quad (11)$$

Similarly,

$$\frac{d^2 y}{dt^2} = 2 \frac{d^2 \hat{y}}{dt^2} \quad (12)$$

The nonlinear term transforms as:

$$y^2 = 4\hat{y}^2 \quad (13)$$

The factor of 4 introduced in the nonlinear term must be compensated within the feedback structure. Accordingly:

- The output of the multiplier block is scaled by 4,
- The upper input to the summation block feeding the leftmost integrator is compensated appropriately,
- Gain redistribution is performed across the feedback path.

Amplitude scaling is important because analog computing blocks and numerical solvers operate more reliably when signals remain within bounded operating ranges. It prevents saturation in integrators and improves numerical stability during nonlinear oscillations.

b) *Scaling of the Nonlinear Term:* The squared signal $\hat{y}^2$ lies within the interval:

$$0 \leq \hat{y}^2 \leq 1 \quad (14)$$

Without scaling, the original nonlinear term y^2 would vary between 0 and 4. Therefore, scaling adjustments were redistributed throughout the computational graph to maintain equivalent system dynamics while keeping internal signals bounded.

The scale factor introduced after the multiplier was redistributed to neighboring summation and gain blocks, thereby preserving the original equation while improving practical implementability.

c) *Time Scaling:* Time scaling was introduced to slow down the oscillator dynamics and stabilize the integrator chain. The scaled time variable was defined as:

$$\tau = \frac{t}{10} \quad (15)$$

Therefore,

$$\frac{d\tau}{dt} = \frac{1}{10} \quad (16)$$

Applying the chain rule:

$$\frac{d\hat{y}}{dt} = \frac{d\hat{y}}{d\tau} \frac{d\tau}{dt} = \frac{1}{10} \frac{d\hat{y}}{d\tau} \quad (17)$$

Similarly,

$$\frac{d^2 \hat{y}}{dt^2} = \frac{1}{100} \frac{d^2 \hat{y}}{d\tau^2} \quad (18)$$

Time scaling reduces rapid state transitions and distributes gain more effectively across the integrator chain. It also improves solver convergence and prevents unstable oscillatory growth during simulation.

Substituting the scaled variables into the Van der Pol equation yields:

$$\hat{y}'' = 10\mu(1 - 4\hat{y}^2)\hat{y}' - 100\hat{y} \quad (19)$$

The scaling procedure introduces several gain constants throughout the analog implementation:

- Gain factor 10 in the integrator chain due to time scaling,
- Gain factor $\frac{1}{5}$ in the feedback branches,
- Gain factor $\frac{1}{4}$ associated with nonlinear compensation,
- Gain factor $\frac{4}{5}$ introduced during redistribution of nonlinear scaling terms.

Executing the scaling process throughout the complete computational structure produces the following auxiliary equations:

$$-\dot{\hat{y}} = -\frac{10}{4} \int \ddot{\hat{y}} d\tau \quad (20)$$

$$\hat{y} = -2 \int -\dot{\hat{y}} d\tau \quad (21)$$

$$y_1 = -\left(-\frac{1}{4} + \hat{y}^2\right) \quad (22)$$

$$y_2 = -\dot{\hat{y}} y_1 \quad (23)$$

$$y_3 = \frac{4}{5} y_2 \mu \quad (24)$$

$$\ddot{\hat{y}} = -\frac{\hat{y}}{5} + 10y_3 \quad (25)$$

Figure 2 shows the final scaled analog computer setup of the Van der Pol oscillator after amplitude scaling, time scaling, and gain redistribution.

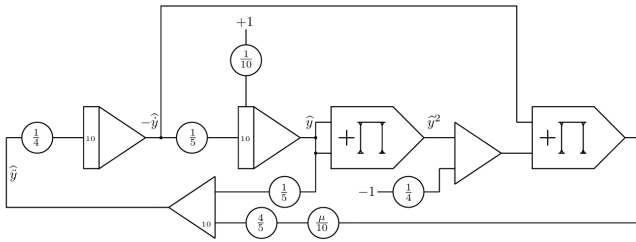


Fig. 2. Scaled setup of Van der pol oscillator

In the scaled architecture, the integrator blocks contain gain factors of 10 introduced due to time scaling. Additional gain blocks such as $\frac{1}{4}$, $\frac{1}{5}$, and $\frac{4}{5}$ compensate for amplitude normalization and nonlinear term redistribution. The multiplier block generates the nonlinear term $\hat{y}^2$, while the feedback structure reconstructs the scaled nonlinear damping dynamics.

3) *Simulink Implementation of the Scaled Van der Pol Oscillator:* After applying amplitude scaling, time scaling, and gain redistribution, the scaled Van der Pol oscillator was implemented in MATLAB Simulink using an analog computing inspired architecture. The implementation reproduces the nonlinear differential equation through interconnected subsystems representing integration, summation, nonlinear multiplication, and gain operations.

Figure 3 shows the complete Simulink implementation of the scaled Van der Pol oscillator.

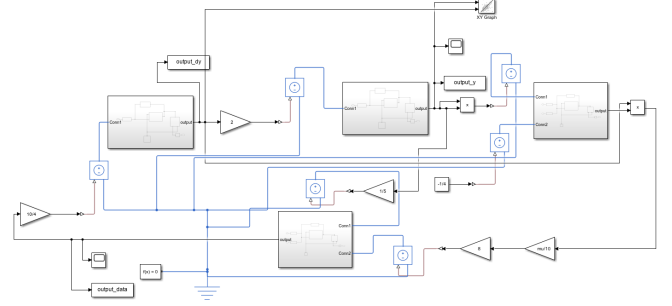


Fig. 3. Simulink setup of the scaled oscillator

The Simulink architecture consists of:

- Integrator subsystems,
- Summer subsystems,
- Gain blocks,
- Product blocks for nonlinear multiplication,
- Feedback loops implementing nonlinear damping,
- To Workspace blocks for data extraction.

The system implements the scaled Van der Pol equation:

$$\hat{y}'' = 10\mu(1 - 4\hat{y}^2)\hat{y}' - 100\hat{y} \quad (26)$$

The gain blocks containing factors such as 10, $\frac{1}{5}$, $\frac{1}{4}$, and $\frac{\mu}{10}$ arise directly from the scaling procedure discussed previously. The multiplier blocks generate the nonlinear term $\hat{y}^2$, while the feedback structure reconstructs the nonlinear damping expression.

a) *Integrator Subsystem:* The integrator subsystem shown in Figure 4 was implemented using Simscape electrical components and operational amplifier based feedback integration.

The subsystem performs continuous-time integration of the input signal. Capacitive feedback is used to realize the integration operation, while the operational amplifier maintains the required feedback dynamics. The output voltage corresponds to the integrated signal state variable.

Two cascaded integrator blocks are used in the main architecture:

- First integrator: generates $\dot{y}$ from $\ddot{y}$,
- Second integrator: generates y from $\dot{y}$.

The integrator chain converts the second-order differential equation into interconnected first-order dynamical systems suitable for analog realization.

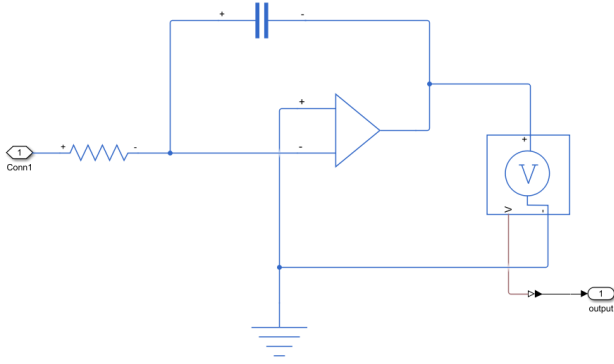


Fig. 4. The integrator subsystem

b) *Summer Subsystem*: The summation subsystem shown in Figure 5 combines multiple feedback signals required to reconstruct the Van der Pol dynamics.

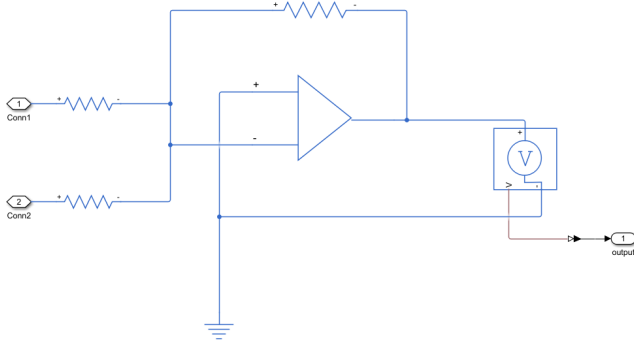


Fig. 5. The summer subsystem

The summer subsystem uses operational amplifier based summation to combine:

- Linear restoring force components,
- Nonlinear damping feedback,
- Scaling compensation terms.

This subsystem plays a critical role in implementing the feedback structure governing the nonlinear oscillatory behavior.

B. Dataset Generation Pipeline

A complete dataset generation framework was developed using MATLAB and Simulink simulations of the Van der Pol oscillator. The objective of the pipeline was to generate labeled oscillatory signal data corresponding to different nonlinear regimes for subsequent signal analysis and machine learning based classification.

1) *Generation of Oscillatory Regimes*: Different oscillatory behaviors were generated by varying the nonlinear damping parameter μ . The selected values were grouped into four distinct oscillation regimes:

TABLE I
OSCILLATION REGIMES BASED ON μ

Regime	μ Values
Harmonic Region	0, 10^{-10} , 10^{-9} , 10^{-8} , 10^{-7}
Weak Nonlinear Region	0.01, 0.05, 0.1, 0.2, 0.5
Strong Nonlinear Region	2, 3, 4, 5, 7
Relaxation Oscillation Region	10, 12, 15, 18, 20

Small values of μ generate nearly sinusoidal harmonic oscillations, whereas larger values introduce stronger nonlinear damping effects and relaxation oscillatory behavior.

2) *Simulation Configuration*: The Simulink model was executed repeatedly for all selected μ values using the following simulation parameters:

TABLE II
SIMULATION PARAMETERS

Parameter	Value
Solver	ODE45
Solver Type	Variable-step
Stop Time	0.2 s

The ODE45 variable-step solver was selected because of its effectiveness in solving nonlinear ordinary differential equations with varying oscillatory behavior.

3) *Signal Extraction*: For each simulation, the following signals were extracted using To Workspace blocks:

TABLE III
EXTRACTED SIGNALS

Signal	Description
y	Oscillator displacement
$\dot{y}$	Velocity signal
$\ddot{y}$	Acceleration signal

These signals collectively capture the nonlinear dynamical behavior of the oscillator.

4) *Signal Validation*: Before further processing, each generated signal was validated to ensure numerical correctness and simulation stability. The simulation output was rejected if:

- The signal was empty,
- NaN values were present,
- Infinite values were present.

This validation stage prevented corrupted or unstable simulations from entering the dataset.

5) *Removal of Initial Transient*: The initial portion of the oscillator response contains transient dynamics that do not represent steady-state oscillatory behavior. Therefore, the first 20% of the generated samples were removed.

Removing transient regions ensures that extracted features represent stable nonlinear oscillations rather than startup dynamics.

6) *Sliding Window Segmentation*: After transient removal, the continuous oscillatory signals were segmented using a sliding-window approach.

The segmentation parameters were:

TABLE IV
SLIDING WINDOW PARAMETERS

Parameter	Value
Window Size	100 samples
Stride	50 samples

For each iteration:

- 100 consecutive samples were selected,
- The window was shifted by 50 samples,
- The next overlapping segment was extracted.

The use of overlapping windows increases dataset size while preserving local oscillatory behavior.

The necessity of windowing includes:

- Increasing dataset size from limited simulations,
- Capturing local nonlinear signal dynamics,
- Reducing computational complexity during feature extraction,
- Improving learning performance of machine learning models.

7) *Window Normalization*: Each displacement window was normalized using z-score normalization:

$$y_{norm} = \frac{y - \mu_y}{\sigma_y + \epsilon} \quad (27)$$

where:

- μ_y = mean of the window,
- σ_y = standard deviation,
- ϵ = small constant preventing division by zero.

Normalization removes scale variations between windows and improves feature consistency across different oscillatory regimes.

8) *Regime Labeling*: Each segmented window was assigned a class label based on the corresponding value of μ .

TABLE V
OSCILLATION REGIME LABELS

Condition	Label
$\mu < 10^{-7}$	Harmonic
$10^{-7} < \mu < 1$	Weak_Nonlinear
$1 \leq \mu < 10$	Strong_Nonlinear
$\mu \geq 10$	Relaxation

This created a supervised learning dataset suitable for classification tasks.

9) *Final Dataset Construction*: For each segmented oscillatory window:

- Statistical features,
- Energy-based features,
- Nonlinear signal descriptors

were extracted and stored along with their corresponding regime labels.

The final dataset consisted of 16,514 samples generated across multiple nonlinear regimes. The extracted features and labels were stored in MATLAB table format and exported as a CSV dataset for machine learning analysis.

C. Feature Extraction

Feature extraction was performed to convert raw oscillatory signals into compact numerical representations describing waveform shape, oscillation intensity, energy distribution, and nonlinear complexity. These features enable machine learning models to distinguish between different oscillatory regimes.

The extracted features were computed for each segmented signal window.

1) Variability Feature:

a) *Standard Deviation*: Standard deviation measures the spread or variability of the signal around its mean.

$$\sigma = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2} \quad (28)$$

where:

- x_i = signal sample,
- $\bar{x}$ = mean of the signal,
- N = total number of samples.

Higher values indicate larger oscillation variations.

2) RMS Features:

a) *Root Mean Square (RMS)*: RMS measures the effective magnitude of the oscillatory signal.

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (29)$$

RMS features were computed for:

- Displacement signal (y),
- Velocity signal ($\dot{y}$),
- Acceleration signal ($\ddot{y}$).

These features represent oscillation intensity and signal energy.

3) Energy Features:

a) *Signal Energy*: Signal energy measures the total strength of the signal.

$$E = \sum_{i=1}^N x_i^2 \quad (30)$$

b) *Signal Power*: Signal power represents the average energy per sample.

$$P = \frac{1}{N} \sum_{i=1}^N x_i^2 \quad (31)$$

Higher energy and power values indicate stronger oscillatory activity.

4) Amplitude Features:

a) *Peak-to-Peak Value*: The peak-to-peak value measures the total amplitude swing.

$$PP = \max(x) - \min(x) \quad (32)$$

b) *Crest Factor*: Crest factor measures the sharpness of waveform peaks.

$$CF = \frac{\max |x|}{RMS} \quad (33)$$

Relaxation oscillations generally exhibit higher crest factors because of rapid transitions and sharp waveform peaks.

5) *Oscillation Feature*:

a) *Zero Crossing Rate (ZCR)*: Zero Crossing Rate measures how frequently the signal crosses zero.

$$ZCR = \sum_{i=1}^{N-1} |sgn(x[i]) - sgn(x[i-1])| \quad (34)$$

where:

$$sgn(x) = \begin{cases} 1, & x \geq 0 \\ -1, & x < 0 \end{cases} \quad (35)$$

Higher zero-crossing counts indicate faster oscillatory behavior.

6) *Entropy Feature*:

a) *Entropy*: Entropy measures signal randomness and nonlinear complexity.

$$H = - \sum p_i \log_2(p_i) \quad (36)$$

where p_i represents the probability distribution of signal amplitudes.

Higher entropy values indicate stronger nonlinear and complex oscillatory dynamics.

7) *Shape Features*:

a) *Skewness*: Skewness measures waveform asymmetry.

$$Skewness = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^3}{\sigma^3} \quad (37)$$

Positive skewness indicates right-skewed waveforms, while negative values indicate left-skewed waveforms.

b) *Kurtosis*: Kurtosis measures waveform sharpness and peak concentration.

$$Kurtosis = \frac{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^4}{\sigma^4} \quad (38)$$

Higher kurtosis values indicate sharper peaks and stronger impulsive behavior.

8) *Additional Features*:

a) *Mean Absolute Value (MAV)*: Mean Absolute Value represents the average amplitude level.

$$MAV = \frac{1}{N} \sum_{i=1}^N |x_i| \quad (39)$$

b) *Shape Factor*: Shape factor describes waveform shape characteristics.

$$SF = \frac{RMS}{MAV} \quad (40)$$

Different oscillation regimes exhibit distinct shape-factor distributions due to changes in waveform morphology.

The extracted feature set collectively captures:

- Oscillation magnitude,
- Nonlinear waveform characteristics,
- Energy distribution,
- Signal complexity,
- Oscillation frequency behavior.

D. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed to study the statistical properties of the generated oscillatory dataset and understand how different nonlinear regimes vary with respect to extracted signal features. The following analyses were performed:

- Distribution of generated samples for each μ value,
- Class distribution across oscillation regimes,
- Mean RMS variation with increasing μ ,
- Entropy variation across nonlinear regimes,
- Crest factor analysis,
- Kurtosis trend analysis,
- Correlation heatmap of extracted features,
- Boxplot visualization of important features.

E. Machine Learning Preprocessing

Several preprocessing steps were performed before machine learning model training to improve generalization capability, prevent data leakage, and ensure consistent feature representation.

1) *Dataset Splitting Based on μ Values*: The dataset was divided using distinct μ values instead of random sample-wise splitting.

The training dataset used the following μ values:

- Harmonic Region:

$$0, 10^{-10}, 10^{-9}$$

- Weak Nonlinear Region:

$$0.01, 0.05, 0.1$$

- Strong Nonlinear Region:

$$2, 3, 4$$

- Relaxation Region:

$$10, 12, 15$$

The testing dataset used previously unseen μ values:

- Harmonic Region:

$$10^{-8}, 10^{-7}$$

- Weak Nonlinear Region:

0.2, 0.5

- Strong Nonlinear Region:

5, 7

- Relaxation Region:

18, 20

2) *Avoiding Data Leakage*: The use of μ -based splitting was essential to avoid data leakage. The dataset was generated using overlapping sliding windows from continuous oscillator simulations. Random sample-wise splitting could cause highly similar windows from the same simulation trajectory to appear in both training and testing datasets. This would artificially increase classification accuracy and reduce the reliability of model evaluation.

By separating train and test datasets using distinct μ values:

- Training and testing samples originated from different oscillator conditions,
- Models were evaluated on unseen nonlinear regimes,
- Generalization capability was assessed more realistically.

3) *Training Dataset Distribution Before Balancing*: The original training dataset contained unequal sample distributions because higher nonlinear regions produced more oscillatory windows. The sample distribution before balancing was:

TABLE VI
TRAINING SAMPLES BEFORE BALANCING

μ Value	Samples
0	354
10^{-10}	354
10^{-9}	354
0.01	375
0.05	373
0.1	372
2	780
3	900
4	1006
10	1294
12	1378
15	1588

4) *Balanced Training Dataset*: To reduce model bias toward dominant nonlinear regimes, only the training dataset was balanced. The minimum sample count among all training μ values was:

$$N_{min} = 354 \quad (41)$$

Therefore, 354 samples were randomly selected from each training μ value.

The balanced training dataset distribution became:

TABLE VII
BALANCED TRAINING DATASET

Oscillation Regime	Samples
Harmonic	1062
Weak_Nonlinear	1062
Strong_Nonlinear	1062
Relaxation	1062

The final balanced training dataset contained:

$$4248 \text{ samples} \quad (42)$$

The testing dataset was intentionally left unbalanced to preserve the natural distribution of nonlinear oscillatory behavior for realistic model evaluation.

The final testing dataset contained:

$$7386 \text{ samples} \quad (43)$$

5) *Feature Scaling*: The extracted features had different numerical ranges and physical interpretations. Therefore, feature standardization was performed using StandardScaler.

Each feature was transformed as:

$$x_{scaled} = \frac{x - \mu}{\sigma} \quad (44)$$

where:

- μ = feature mean,
- σ = feature standard deviation.

Feature scaling is important because:

- Distance-based models such as KNN and SVM are sensitive to feature magnitude,
- Neural networks converge faster with normalized features,
- PCA requires standardized features for meaningful variance analysis.

6) *Principal Component Analysis (PCA)*: Principal Component Analysis (PCA) was applied for dimensionality reduction and visualization of feature separability. PCA transforms the original feature space into orthogonal principal components that maximize variance representation. The first two principal components were used for visualization of oscillation regime clustering.

The PCA analysis demonstrated clear separability among:

- Harmonic oscillations,
- Weak nonlinear oscillations,
- Strong nonlinear oscillations,
- Relaxation oscillations.

The first two principal components captured a major portion of the dataset variance, indicating that the extracted statistical and nonlinear features effectively represented the underlying oscillator dynamics.

F. Training and Evaluation Metrics

Multiple machine learning models were evaluated for oscillation regime classification using the extracted statistical and nonlinear features. A diverse set of models was selected to compare the effectiveness of linear classifiers, probabilistic methods, kernel-based approaches, neural networks, and ensemble learning techniques for nonlinear oscillatory pattern recognition.

The following models were used:

- Ridge Classifier
- Logistic Regression
- Linear Discriminant Analysis (LDA)

- Stochastic Gradient Descent (SGD) Classifier
- Passive Aggressive Classifier
- Linear Support Vector Machine (Linear SVM)
- Nearest Centroid
- Multi-Layer Perceptron (MLP – Shallow)
- Multi-Layer Perceptron (MLP – Wide)
- Multi-Layer Perceptron (MLP – Deep)
- Multinomial Naive Bayes
- Quadratic Discriminant Analysis (QDA)
- Bernoulli Naive Bayes
- Nu-Support Vector Classifier (NuSVC – RBF)
- Histogram Gradient Boosting
- Support Vector Machine (SVM – RBF)
- Extra Trees Classifier
- K-Nearest Neighbors (KNN)
- Random Forest
- Gaussian Naive Bayes
- Gradient Boosting
- Bagging Classifier (Decision Tree)
- Perceptron
- Decision Tree
- AdaBoost

1) *Hyperparameter Optimization Using Grid Search:* To improve classification performance, optimal hyperparameters for each model were determined using the Grid Search optimization algorithm. Grid Search performs an exhaustive search over predefined hyperparameter combinations and identifies the configuration producing the best validation performance.

The optimization process included tuning parameters such as:

- Regularization strength,
- Learning rate,
- Kernel type,
- Number of estimators,
- Tree depth,
- Number of neighbors,
- Hidden layer architecture,
- Activation functions,
- Splitting criteria,
- Distance metrics.

The optimized hyperparameters obtained through Grid Search were used for final model training and evaluation.

2) *Justification for Model Selection:* The selected models represent multiple categories of machine learning approaches:

- Linear models: Ridge Classifier, Logistic Regression, SGD Classifier, Passive Aggressive, Linear SVM, Perceptron
- Probabilistic models: Gaussian NB, Bernoulli NB, Multinomial NB
- Discriminant analysis methods: LDA, QDA
- Distance-based methods: KNN, Nearest Centroid
- Kernel-based nonlinear classifiers: SVM (RBF), NuSVC (RBF)
- Neural network architectures: MLP (Shallow), MLP (Wide), MLP (Deep)

- Tree-based and ensemble learning methods: Decision Tree, Random Forest, Extra Trees, Bagging, AdaBoost, Gradient Boosting, Histogram Gradient Boosting

This broad model selection enabled comprehensive comparison of different learning paradigms for nonlinear oscillatory signal classification.

3) *Evaluation Metrics:* The machine learning models were evaluated using multiple performance metrics to analyze classification capability, class-wise behavior, probabilistic discrimination performance, and computational efficiency.

The following evaluation metrics were used:

- Cross-Validated Weighted F1-Score
- Accuracy
- Weighted Precision
- Weighted Recall
- Weighted F1-Score
- Macro Precision
- Macro Recall
- Macro F1-Score
- One-vs-Rest ROC-AUC (`roc_auc_ovr`)
- Model Training Time (`fit_time_s`)
- Prediction Time (`predict_time_s`)

III. RESULTS AND ANALYSIS

A. Time-Domain and Phase-Plane Analysis

The oscillatory behavior of the Van der Pol oscillator was analyzed in both the time domain and phase plane for different values of the nonlinear parameter μ . The analysis illustrates the transition from linear harmonic oscillations to strongly nonlinear relaxation oscillations as μ increases.

1) *Time-Domain Oscillation Analysis:* Figure 6 shows the displacement response of the Van der Pol oscillator for four representative nonlinear regimes:

- $\mu = 0$
- $\mu = 2$
- $\mu = 5$
- $\mu = 12$

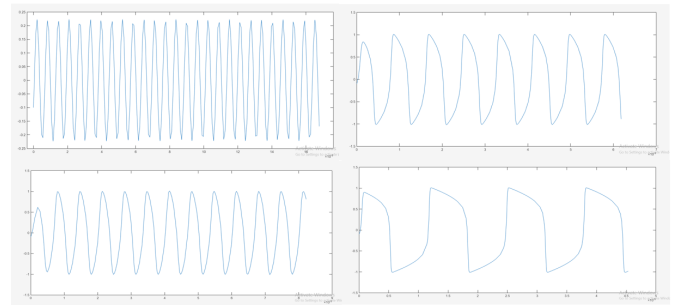


Fig. 6. Time domain analysis

For $\mu = 0$, the oscillator behaves as a linear harmonic oscillator with smooth sinusoidal oscillations and constant amplitude. The waveform is symmetric and periodic because no nonlinear damping is present.

For $\mu = 2$, nonlinear effects become noticeable. The oscillations remain periodic but begin to exhibit waveform distortion caused by nonlinear damping.

At $\mu = 5$, stronger nonlinear behavior appears, producing asymmetric oscillations and non-uniform waveform evolution. The periodic motion deviates significantly from the sinusoidal shape observed in the linear regime.

For $\mu = 12$, the oscillator exhibits relaxation oscillations characterized by slow state evolution followed by rapid transitions. The waveform becomes highly distorted and nearly rectangular in shape due to strong nonlinear damping.

These results demonstrate the progressive transition from harmonic oscillation to relaxation oscillation as the nonlinear parameter μ increases.

2) *Phase-Plane Analysis*: Phase-plane trajectories were generated using displacement (y) and velocity ($\dot{y}$) signals extracted from the Simulink model. The phase plots provide insight into system stability, nonlinear damping behavior, and limit cycle formation.

a) *Case 1: $\mu = 0$ (Linear Oscillator)*: Figure 7 shows the phase-plane trajectory for $\mu = 0$.

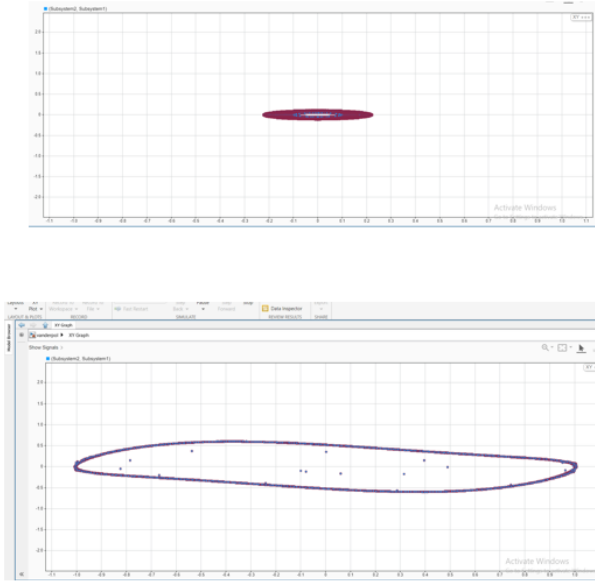


Fig. 7. Phase plane analysis

For $\mu = 0$, the phase trajectory forms a nearly perfect elliptical orbit corresponding to simple harmonic motion. The trajectory neither converges nor diverges, indicating that energy is conserved in the system. Since no nonlinear damping is present, the oscillator behaves like a linear conservative system and no stable limit cycle exists.

b) *Case 2: $\mu = 1$ (Moderate Nonlinearity)*: For $\mu = 1$, the trajectory initially starts near the origin and gradually spirals outward before converging to a closed-loop trajectory. This closed loop corresponds to a stable limit cycle.

The origin acts as an unstable equilibrium point, meaning that small disturbances move the system away from

equilibrium. As the oscillation amplitude increases, nonlinear damping regulates the energy exchange process and stabilizes the trajectory into sustained periodic oscillations.

c) *Case 3: $\mu = 2$ (Stronger Nonlinearity)*: Figure 8 shows the phase-plane trajectory for $\mu = 2$.

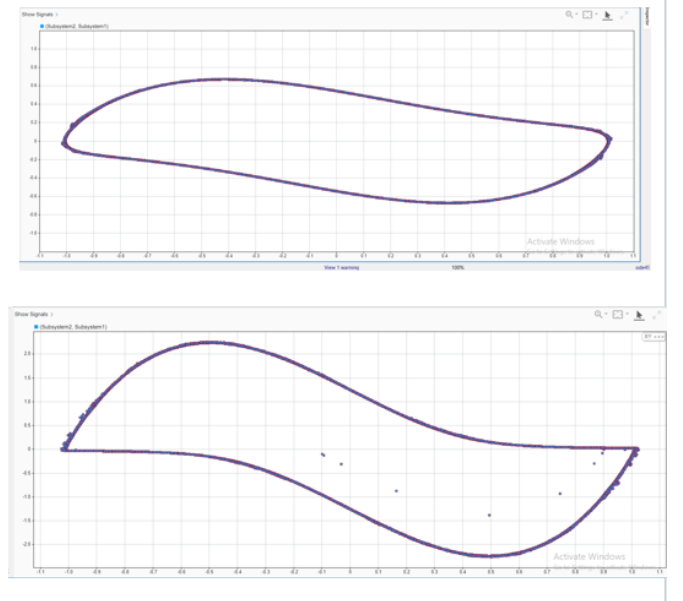


Fig. 8. Phase plot analysis

For $\mu = 2$, the limit cycle becomes sharper and more elongated compared to the nearly elliptical trajectory observed for smaller μ values. The trajectories converge more rapidly toward the stable limit cycle because stronger nonlinear damping quickly suppresses deviations from the periodic orbit.

The oscillatory motion becomes increasingly non-uniform, indicating the growing dominance of nonlinear effects. Energy is injected rapidly at small amplitudes and dissipated more strongly at larger amplitudes.

d) *Case 4: $\mu = 12$ (High Nonlinearity)*: For $\mu = 12$, the phase trajectory becomes highly distorted and nearly rectangular in shape. The oscillator exhibits strong relaxation oscillations characterized by alternating regions of:

- Slow state evolution,
- Rapid transitions between states.

The trajectory contains sharp corners and abrupt directional changes, indicating strong nonlinear damping and highly non-sinusoidal oscillatory behavior.

The phase-plane analysis clearly demonstrates the transition from:

- Linear harmonic oscillations,
 - Stable nonlinear limit cycles,
 - Strong relaxation oscillations
- as the nonlinear parameter μ increases.

B. Exploratory Data analysis

Exploratory Data Analysis (EDA) was performed to analyze how the extracted statistical and nonlinear features vary across different oscillatory regimes of the Van der Pol oscillator.

1) *RMS Variation Across μ* : Figure 9 shows the variation of mean RMS values with respect to the nonlinear parameter μ . The RMS values remain nearly constant across different

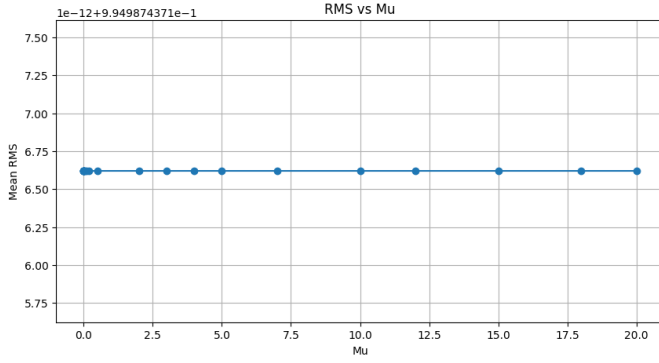


Fig. 9. RMS vs μ

μ values, indicating that the overall signal energy remains relatively stable despite increasing nonlinearity. This suggests that changes in oscillatory behavior are primarily reflected in waveform morphology rather than overall signal magnitude.

2) *Entropy Variation Across μ* : Figure 10 illustrates the variation of mean entropy values with increasing μ . Entropy

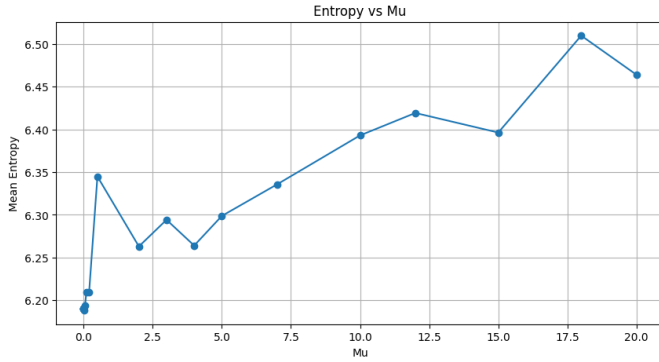


Fig. 10. Entropy vs μ

gradually increases as μ increases, particularly in strong nonlinear and relaxation oscillation regions. This indicates increasing waveform complexity and stronger nonlinear behavior at higher μ values. Relaxation oscillations exhibit the highest entropy because of abrupt state transitions and highly distorted waveforms.

3) *Crest Factor Variation Across μ* : Figure 11 shows the variation of crest factor with respect to μ . The crest factor exhibits noticeable variations across oscillatory regimes. Harmonic and weak nonlinear regions maintain relatively higher crest factors, while strong nonlinear regions exhibit lower values. Relaxation oscillations show larger fluctuations because of rapid waveform transitions and sharp oscillatory peaks.

4) *Kurtosis Variation Across μ* : Figure 12 illustrates the variation of kurtosis across different values of μ . Kurtosis

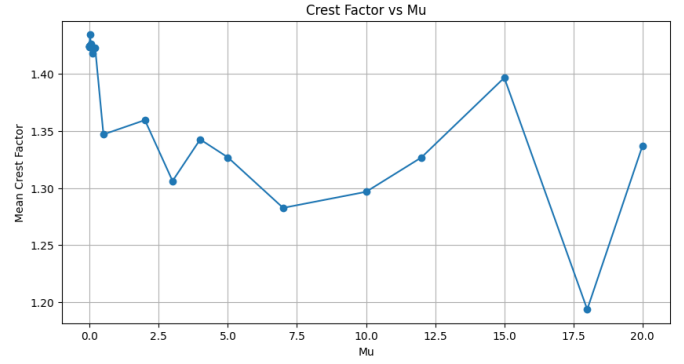


Fig. 11. Crest factor vs μ

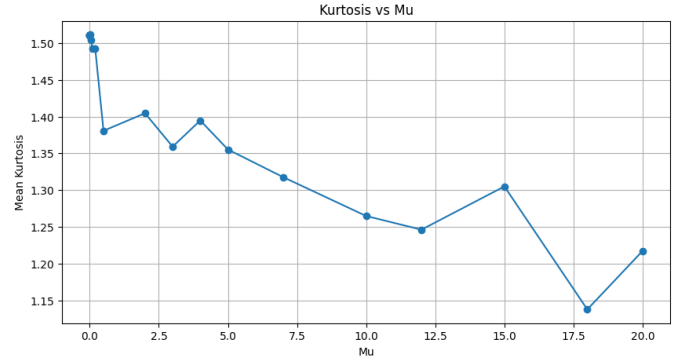


Fig. 12. Kurtosis vs μ

decreases as the oscillator transitions from harmonic to relaxation regimes. Harmonic oscillations exhibit sharper and more concentrated waveform peaks, whereas relaxation oscillations produce broader distributions with more spread-out waveform structures.

5) *Feature Correlation Analysis*: The correlation heatmap shown in Figure 13 was generated to analyze relationships among extracted features. Several strong correlations were observed:

- RMS and standard deviation features show very high positive correlation,
- Energy and signal power are strongly correlated,
- Entropy exhibits strong negative correlation with kurtosis and shape factor,
- Peak-to-peak amplitude strongly correlates with zero-crossing behavior,
- Mean absolute value and shape factor exhibit strong inverse correlation.

These relationships indicate that different features capture complementary aspects of nonlinear oscillatory dynamics.

6) *Entropy Distribution Across Regimes*: Figure 14 shows the entropy distribution across oscillatory regimes.

Relaxation oscillations exhibit the highest entropy distribution, confirming their increased nonlinear complexity. Harmonic and weak nonlinear regimes exhibit lower entropy values because of smoother oscillatory behavior.

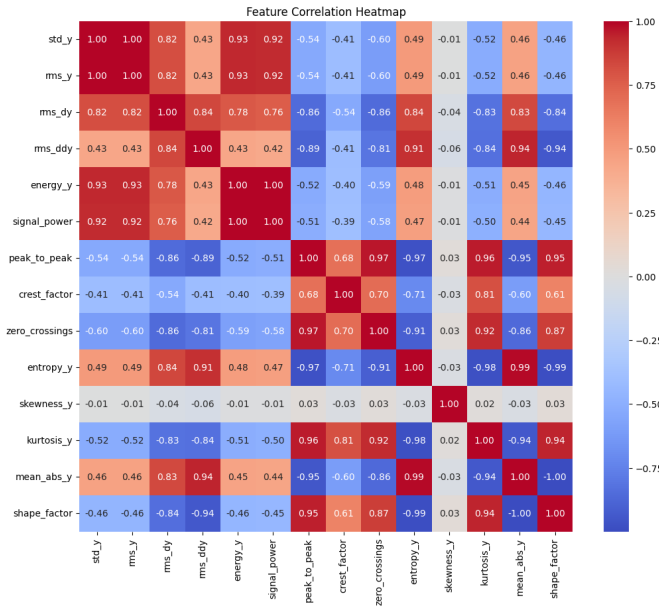


Fig. 13. Correlation heatmap

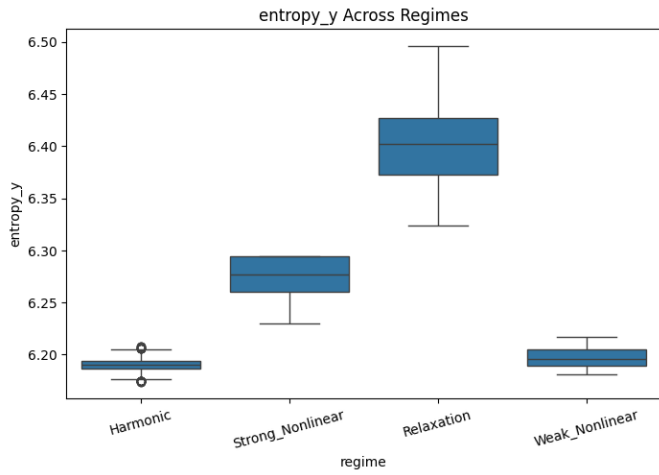


Fig. 14. Entropy distribution across regimes

7) *Crest Factor Distribution Across Regimes*: Figure 15 shows the crest factor distribution across oscillation regimes. Relaxation oscillations exhibit the widest crest factor distribution because of sudden waveform transitions and irregular oscillatory peaks. Harmonic oscillations maintain more stable crest factor behavior.

8) *Kurtosis Distribution Across Regimes*: Figure 16 shows kurtosis distributions across oscillation regimes. Harmonic and weak nonlinear regimes exhibit higher kurtosis values, indicating sharper waveform peaks. Relaxation oscillations exhibit lower kurtosis because of broader oscillatory transitions.

9) *PCA Visualization of Oscillation Regimes*: Principal Component Analysis (PCA) was performed to visualize feature separability among oscillatory regimes. Figure 17 shows the projection of samples onto the first two principal com-

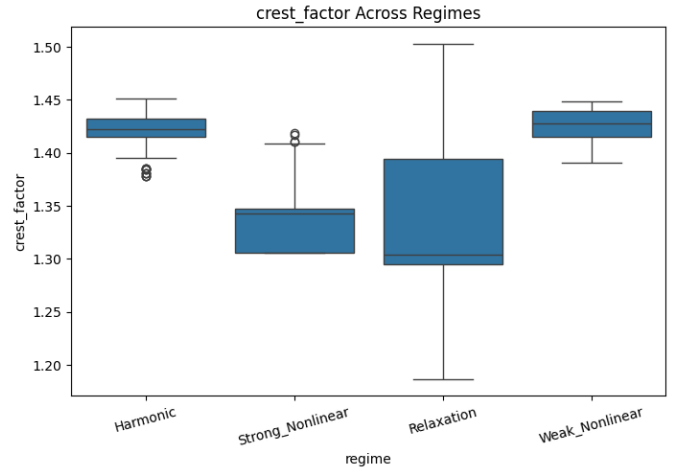


Fig. 15. Crest factor distribution across regimes

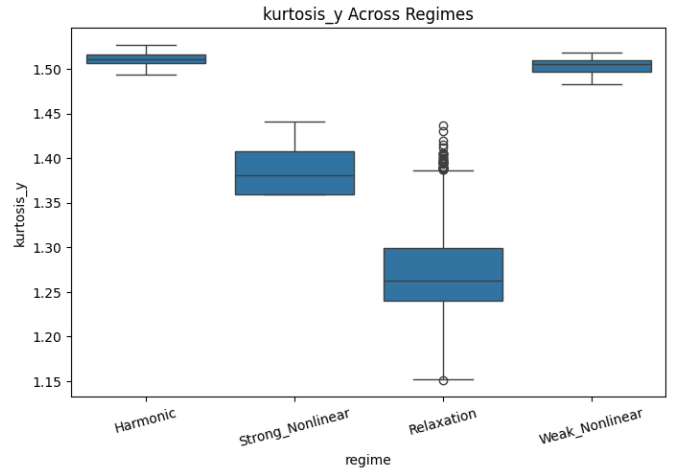


Fig. 16. Kurtosis distribution across regimes

ponents. The PCA visualization demonstrates clear clustering

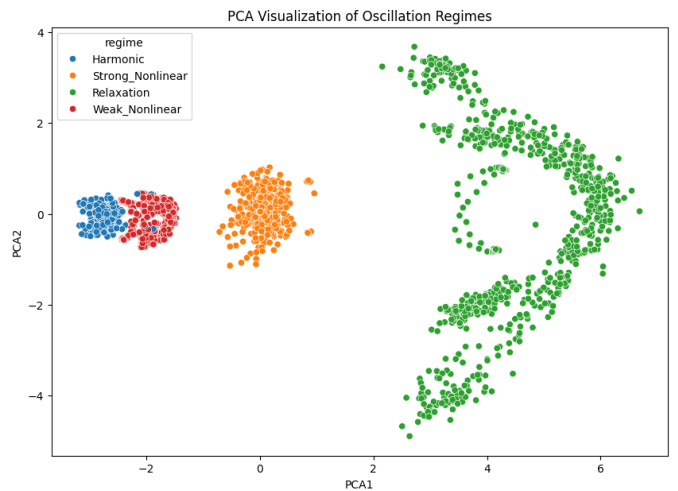


Fig. 17. PCA analysis

among Harmonic oscillations, Weak nonlinear oscillations, Strong nonlinear oscillations and Relaxation oscillations. Relaxation oscillations form a highly distinct cluster because of their strong nonlinear dynamics and waveform complexity. The explained variance ratios of the first two principal components were:

$$PCA_1 = 78.35\% \quad (45)$$

$$PCA_2 = 10.11\% \quad (46)$$

Thus, the first two principal components together captured:

$$88.45\% \quad (47)$$

of the total dataset variance, indicating that the extracted features effectively represent the underlying nonlinear oscillator dynamics.

C. Machine Learning Evaluation Results

The performance of the selected machine learning models was evaluated using the extracted statistical and nonlinear features obtained from the Van der Pol oscillator dataset. The models were compared using weighted F1-score, accuracy, precision, recall, ROC-AUC, training time, and prediction time metrics.

1) *Overall Model Performance:* Table VIII summarizes the performance of the evaluated machine learning models.

The results indicate that linear classification models performed exceptionally well on the extracted oscillatory features. Among all evaluated models, the Ridge Classifier achieved the highest overall classification performance with an accuracy of approximately 99.7%, indicating strong separability of oscillatory regimes in the extracted feature space.

Logistic Regression and Linear Discriminant Analysis (LDA) also demonstrated strong performance, achieving accuracies above 95%. Their high ROC-AUC scores indicate excellent discrimination capability between oscillation regimes.

2) *Performance of Linear Models:* Linear models such as Ridge Classifier, Logistic Regression, LDA, SGD Classifier, Linear SVM performed consistently well across all evaluation metrics. This suggests that the extracted statistical and nonlinear features provide strong linear separability between harmonic, weak nonlinear, strong nonlinear, and relaxation oscillation regimes. The Ridge Classifier achieved the best trade-off between classification accuracy and computational efficiency, while LDA provided very fast training with high classification capability.

3) *Performance of Neural Network Models:* The neural network architectures: MLP (Shallow), MLP (Wide), MLP (Deep) achieved moderate classification performance compared to linear and ensemble models. Although the neural networks achieved high ROC-AUC values, their training times were significantly larger. The wide and deep MLP architectures required substantially higher computational cost without corresponding improvements in classification accuracy. This

indicates that the extracted feature space was already sufficiently separable, reducing the need for deeper nonlinear representations.

4) *Performance of Distance-Based and Probabilistic Models:* Distance-based models such as KNN, Nearest Centroid showed moderate classification performance. Their results indicate that local feature similarity exists among oscillatory regimes, but nonlinear overlaps reduce neighborhood-based classification efficiency. Probabilistic models including Gaussian Naive Bayes, Bernoulli Naive Bayes, Multinomial Naive Bayes showed comparatively lower performance because the extracted features do not strictly satisfy the independence assumptions of Naive Bayes classifiers.

5) *Performance of Ensemble Learning Models:* Tree-based ensemble methods such as Random Forest, Extra Trees, Gradient Boosting, Histogram Gradient Boosting, AdaBoost, Bagging demonstrated stable and reliable performance across all oscillation regimes. These models effectively captured nonlinear feature interactions and provided robust classification performance while maintaining reasonable computational efficiency. Extra Trees and Random Forest particularly showed strong generalization capability because of their ensemble averaging and variance reduction mechanisms.

6) *Computational Analysis:* Training and prediction times were also analyzed to evaluate computational efficiency. LDA and Nearest Centroid exhibited the fastest training times, making them computationally efficient for lightweight implementations. In contrast, deep neural network architectures required significantly larger training times because of increased parameter complexity and iterative optimization procedures. The Ridge Classifier provided an excellent balance between Classification accuracy, Generalization performance, and Computational efficiency.

Overall, the evaluation results demonstrate that the extracted nonlinear oscillatory features effectively separate different Van der Pol oscillation regimes and enable high-performance machine learning based classification.

IV. CONCLUSION

In this work, the Van der Pol oscillator was modeled and analyzed using a MATLAB Simulink based analog computing inspired framework. The nonlinear oscillator dynamics were studied across multiple oscillatory regimes by varying the nonlinear damping parameter μ . The transition from harmonic oscillations to strong nonlinear and relaxation oscillations was successfully observed through time-domain responses and phase-plane analysis.

To enable data-driven analysis, a comprehensive dataset generation pipeline was developed using displacement, velocity, and acceleration signals extracted from Simulink simulations. The generated signals were processed through validation, transient removal, sliding-window segmentation, normalization, and statistical feature extraction. Features such as RMS, entropy, crest factor, kurtosis, zero-crossing rate, and signal energy effectively captured the nonlinear characteristics of different oscillatory regimes.

TABLE VIII
COMPLETE PERFORMANCE COMPARISON OF MACHINE LEARNING MODELS

Model	Accuracy	Weighted F1	Macro F1	ROC-AUC	Training Time (s)
Ridge Classifier	0.9970	0.9970	0.9980	0.9967	2.0563
Logistic Regression	0.9628	0.9621	0.9738	1.0000	10.0529
LDA	0.9580	0.9572	0.9704	1.0000	0.3535
SGD Classifier	0.9395	0.9317	0.8843	0.9900	3.0844
Passive Aggressive	0.9208	0.9128	0.8813	0.9982	1.6720
Linear SVM	0.9041	0.8964	0.8697	1.0000	2.3212
Nearest Centroid	0.8799	0.8707	0.8419	0.9857	0.1335
MLP (Shallow)	0.8771	0.8690	0.8499	0.9988	46.3249
MLP (Wide)	0.8645	0.8558	0.8404	0.9983	143.9200
MLP (Deep)	0.8652	0.8548	0.8315	0.9938	86.0889
Multinomial NB	0.8531	0.8419	0.8212	0.9757	0.6111
QDA	0.8462	0.8343	0.8157	0.9677	0.3166
Bernoulli NB	0.8359	0.8244	0.8488	0.9632	0.6435
NuSVC (RBF)	0.8325	0.8190	0.8045	0.9876	71.1337
Hist Gradient Boosting	0.8069	0.7893	0.7825	0.9493	20.6728
SVM (RBF)	0.8050	0.7870	0.7808	0.9894	49.0869
Extra Trees	0.8002	0.7811	0.7764	0.9928	25.7620
KNN	0.7910	0.7698	0.7679	0.8188	1.3199
Random Forest	0.7882	0.7677	0.7743	0.9634	64.0216
Gaussian NB	0.7870	0.7650	0.7642	0.9606	0.4785
Gradient Boosting	0.7851	0.7637	0.7711	0.9557	132.7733
Bagging (Tree)	0.7836	0.7618	0.7697	0.9651	19.2088
Perceptron	0.7723	0.7468	0.7581	0.9783	1.8471
Decision Tree	0.7203	0.6735	0.6935	0.7699	0.9758
AdaBoost	0.3374	0.1949	0.2316	0.9584	46.2545

A wide range of machine learning models were evaluated for oscillation regime classification. Hyperparameter optimization was performed using the Grid Search optimization algorithm. Among all evaluated models, the Ridge Classifier achieved the highest classification accuracy of 99.70%, demonstrating that the extracted statistical and nonlinear features provide highly discriminative representations of Van der Pol oscillator dynamics.

The proposed framework successfully combines nonlinear dynamical system modeling, analog computation, signal processing, and machine learning for intelligent oscillation regime analysis. The developed methodology can be extended to other nonlinear dynamical systems for automated behavior analysis, anomaly detection, nonlinear system identification, and data-driven classification of complex oscillatory phenomena.

As future work, deep learning architectures and recurrent neural networks can be explored for direct sequence-based oscillation analysis without manual feature extraction. Real-time hardware implementation using analog circuits or FPGA platforms can also be investigated for practical nonlinear system monitoring and adaptive control applications.